

Bases para un cambio de modelo

De llevar el contenido a la aplicación a trabajar con agentes

JA Palazón; jEsuSdA 8)

2026-07-03

Contenido

Antes de empezar	1
Antes de leer	2
1 Introducción	3
2 ¿Qué necesitas hacer?	4
3 ¿Con qué lo haces ahora?	5
4 ¿Qué problemas arrastra ese modelo?	6
5 Un modelo diferente	8
5.1 ¿Qué es OpenCode?	9
5.2 Ejemplo: de un borrador a un informe	9
6 Qué ganas con el cambio	12
6.1 Calidad	12
6.2 Tiempo	12
6.3 Aprendizaje	12
6.4 Independencia	13
7 De la teoría a la práctica	14
7.1 Versionado: olvídate de «versión_final_corregida.docx»	14
7.1.1 Lo que el versionado resuelve	15
7.1.2 Cómo funciona con OpenCode	15
8 Conclusión	16
Referencias	17
8.1 Para empezar	17
8.2 Para profundizar	17
Anexo A: Las plantillas universitarias y el modelo actual	19

Antes de empezar

Este documento describe el modelo de trabajo actual del personal universitario con los contenidos digitales, identifica las deficiencias que arrastra y propone un cambio de enfoque: pasar de depender de aplicaciones concretas a trabajar con contenido en texto plano asistido por agentes de inteligencia artificial. Está pensado como base conceptual para quien se acerca por primera vez a herramientas como OpenCode.

Este documento se dirige principalmente al personal universitario, pero el modelo de trabajo que describe —texto plano como materia prima, agente como transformador— es aplicable a cualquier persona que cree y gestione contenido en su día a día. Estudiantes que preparan trabajos fin de grado, memorias de prácticas o informes de laboratorio. Profesionales en empresas que redactan propuestas, documentan procesos o elaboran presentaciones. Gestores que organizan información, rellenan formularios o mantienen actas. Todos, en la universidad y fuera de ella, compartimos el mismo patrón: el contenido nace, se transforma y se distribuye. Y todos arrastramos los mismos problemas cuando ese contenido depende de la aplicación que lo creó. Si te reconoces en esta descripción —estés donde estés—, este documento también es para ti.

Antes de leer

Si es la primera vez que te acercas a este modelo de trabajo, estos recursos te ayudarán a situarte antes de continuar:

- **Presentación del taller** — Material visual con ejemplos y ejercicios guiados de la sesión práctica.
- **Vídeo de la sesión explicativa** — Grabación de una sesión en la que se explica el modelo y se demuestra el flujo de trabajo con OpenCode.
- **Marcando Palabras** — Libro en desarrollo que amplía estos conceptos con ejercicios prácticos de Markdown y Quarto. Próximamente disponible.

No necesitas verlos todos antes de seguir leyendo. Pero si quieres una idea rápida de cómo funciona todo en la práctica, el vídeo es el recurso más directo.

1 Introducción

Si trabajas en la universidad —docencia, investigación o gestión—, una parte enorme de tu jornada consiste en crear, transformar y reorganizar contenido. Redactas informes, preparas presentaciones, organizas datos en tablas, escribes correos, rellenas solicitudes, elaboras guías docentes, publicas materiales en el aula virtual ...

Para todo eso abres un programa.

Word para el informe. PowerPoint para la clase. Excel para las notas. Google Drive para compartir. Y así, uno tras otro. Cada tarea tiene su herramienta, y cada herramienta tiene su forma de trabajar, sus formatos, sus limitaciones. Te has acostumbrado. Funciona. Pero tiene un coste que apenas notas porque siempre fue así.

El problema de fondo es este: el contenido vive dentro de la aplicación. Lo que escribes en Word se queda en Word. Lo que diseñas en PowerPoint se queda en PowerPoint. Si necesitas la misma información en otro formato, tienes que rehacerla, copiarla a mano o pegar y reajustar. El contenido no es tuyo del todo: es del programa que lo guarda.

Este documento mira ese modelo con ojos nuevos. Identifica qué necesitas hacer realmente, con qué lo haces ahora y qué problemas trae esa dependencia. Y luego propone una forma distinta de trabajar: una donde **el contenido va primero y la aplicación va después**. Donde no necesitas ser experto en cada programa para obtener el resultado que buscas.

No se trata de que dejes de usar Word o PowerPoint. Se trata de que entiendas que hay otra manera, y que esa manera te da más libertad, más calidad y menos pérdidas de tiempo.

2 ¿Qué necesitas hacer?

Antes de hablar de herramientas, hablemos de tareas. En tu día a día universitario, ¿qué tipo de cosas haces con contenido digital? La lista no es corta:

Necesitas ...	¿Para qué?
Redactar documentos extensos	Informes, artículos, guías docentes, memorias, actas
Preparar presentaciones	Clases, seminarios, congresos, defensas de trabajos
Organizar datos en tablas	Calificaciones, presupuestos, resultados de experimentos
Analizar información y crear gráficos	Datos de investigación, encuestas, indicadores
Guardar y recuperar archivos	Tus documentos, tus imágenes, tus borradores
Compartir documentos con otros	Con compañeros, estudiantes, departamentos, revistas
Gestionar referencias y citas	Artículos, libros, fuentes para tus publicaciones
Publicar contenido en web	Asignaturas en Moodle, blogs, sitios de proyecto
Tomar notas rápidas	Ideas, apuntes de reuniones, borradores
Editar imágenes, esquemas o figuras	Para clases, publicaciones, carteles
Comunicarte por escrito	Correos electrónicos, circulares, avisos
Planificar tareas y plazos	Proyectos, entregas, hitos de investigación

Cada una de estas tareas implica manejar información. Y cada una, en el modelo tradicional, requiere una herramienta distinta. El problema no es que haya muchas herramientas: es que **cada herramienta guarda la información en su propio formato**, y esas informaciones no se hablan entre sí.

3 ¿Con qué lo haces ahora?

Si miramos las herramientas que la mayoría de la gente usa realmente para cubrir esas necesidades, el panorama es muy parecido en casi cualquier departamento universitario:

Necesidad	Lo que la mayoría usa
Redactar documentos	Microsoft Word, Google Docs
Preparar presentaciones	Microsoft PowerPoint, Google Slides, Canva
Organizar datos en tablas	Microsoft Excel, Google Sheets
Analizar datos y crear gráficos	Excel, a veces SPSS o programas específicos
Guardar y recuperar archivos	Google Drive, OneDrive, Dropbox, el disco duro o un USB
Compartir documentos	El correo electrónico con archivos adjuntos, Google Drive, Moodle
Gestionar referencias	Zotero, Mendeley ... o una lista a mano, artículo por artículo
Publicar contenido en web	Moodle, WordPress, o no se publica
Tomar notas	El bloc de notas del sistema, Google Keep, una libreta de papel
Editar imágenes	PowerPoint (sí, también para eso), Canva, Photoshop si se tiene
Comunicarse por escrito	Gmail, Outlook
Planificar tareas	El correo, una lista en papel, a veces Trello o similares

Fíjate en un detalle: **casi todas las herramientas son ofimáticas o servicios web**. No hay una estrategia, hay un repertorio. Y cada programa tiene su forma de trabajar, sus menús, sus atajos, sus plantillas. Conoces unas pocas funciones de cada uno —las justas para sobrevivir— y cuando necesitas algo más avanzado, pierdes tiempo buscando cómo se hace o pides ayuda.

No es que estas herramientas sean malas. Es que el modelo de trabajo que imponen **te obliga a saltar de una a otra constantemente**, llevando la información de un lado a otro con copias manuales, pérdidas de formato y trabajo duplicado.

4 ¿Qué problemas arrastra ese modelo?

Este modelo —escribe en Word, pasa a PowerPoint, copia a Excel, adjunta en el correo— es tan común que ni lo cuestionamos. Pero tiene consecuencias concretas que afectan a tu tiempo, a la calidad de tu trabajo y a tu tranquilidad:

El problema	Cómo se nota en tu día a día
Cada programa tiene su formato	Lo que escribes en Word no sirve directamente para una web, una presentación o un PDF. Tienes que rehacerlo o copiarlo y ajustar el formato, siempre con pérdidas.
Los formatos cambian con las versiones	Un documento hecho con Word 2010 no se ve igual en Word 2024. O peor: alguien no puede abrirlo. Tu contenido envejece con el programa.
Las aplicaciones no se hablan entre sí	Copiar una tabla de Excel a PowerPoint parece fácil, hasta que cambias los datos y la diapositiva se queda desactualizada. No hay un hilo que conecte las piezas.
Tu información está secuestrada en la nube	Tus documentos están en Google Drive, OneDrive o Dropbox. Sin conexión no accedes a ellos. Y si la empresa cierra el servicio o cambia las condiciones, tus archivos dependen de su decisión.
Repites la misma tarea una y otra vez	Poner el mismo título en un informe, en la presentación y en el resumen. Ajustar márgenes, numeración, estilos. Son tareas mecánicas que haces a mano cada vez.
La misma información, reescrita para cada destino	Tienes que redactar lo mismo para un artículo, para un informe de departamento, para una comunicación breve y para una entrada en la web. Cuatro veces el mismo contenido, cada vez con su formato.
Pagas por funciones que casi no usas	Las suites ofimáticas cuestan dinero, y la mayoría de la gente usa una fracción muy pequeña de lo que ofrecen. Pagas por tenerlo todo, pero usas muy poco.
Tus datos están en manos de empresas	Cuando usas un servicio web, tus documentos se almacenan en servidores que no controlas. La privacidad de los datos —especialmente los de investigación o gestión— no está garantizada.
Cada actualización te obliga a reaprender	Las interfaces cambian, los menús se reorganizan, las funciones se mueven. Lo que sabías hacer, de repente tienes que volver a buscarlo.
Aprender funciones avanzadas cuesta mucho	Para hacer algo más complejo que lo básico —combinar correspondencia, crear estilos personalizados, automatizar una tarea— necesitas buscar tutoriales, perder tiempo y, a menudo, rendirte.

Precaución

Perder el hilo. Abres un programa para escribir, pero antes de poner la primera frase ya estás ajustando márgenes, cambiando el tipo de letra, alineando elementos. Cambios casi siempre estéticos, que aportan poco al contenido. Pero te han sacado del flujo. La idea que tenías se desvaneció mientras navegabas menús (es el equivalente digital del «doorway effect»¹: al cruzar una puerta, el cerebro segmenta la memoria en episodios y el objetivo anterior se vuelve más difícil de recuperar). Esa interrupción constante —pensar, parar, formatear, retomar— es uno de los costes más invisibles y más altos del modelo actual.

¿Te suena? Son problemas que cualquier universitario reconoce. Pero como siempre fueron así, los asumimos como normales. **No lo son.** Son consecuencia de un modelo de trabajo que ata el contenido a la aplicación. Y ese modelo se puede cambiar.

¹Radvansky, G. A., Krawietz, S. A., & Tamplin, A. K. (2011). Walking through doorways causes forgetting. *Quarterly Journal of Experimental Psychology*, 64(8), 1638–1645.

5 Un modelo diferente

El modelo alternativo parte de una idea muy sencilla: **separa el contenido de la forma**.

En lugar de escribir directamente en Word o PowerPoint, trabajas con tu contenido en un formato que no depende de ningún programa concreto: el texto plano. En concreto, Markdown, una forma de escribir texto con marcas mínimas para indicar títulos, listas, negritas o enlaces.

💡 Así se ve Markdown

Esto es un archivo Markdown. Las marcas son mínimas y se entienden al momento:

```
# Informe de prácticas

## Introducción

Este informe describe las prácticas realizadas en el
departamento de Biología durante el mes de marzo.

## Desarrollo

Los resultados obtenidos fueron:

- Muestra A: crecimiento del 12%
- Muestra B: crecimiento del 8%
- Muestra C: sin datos disponibles

> Los datos deben confirmarse con una segunda ronda
> de mediciones antes de su publicación.

## Conclusiones

El experimento confirma la hipótesis inicial.
Ver [anexo con datos completos](datos.xlsx).
```

Con esto, OpenCode genera un documento Word, una presentación, un PDF o una página web. No necesitas saber nada más de Markdown para empezar.

💡 Tip

No necesitas empezar escribiendo en Markdown desde cero. Puedes dar tus primeras instrucciones a OpenCode sobre documentos .docx o .pdf que ya tengas. Con el uso irás viendo que producir el material directamente en texto plano te da más control, menos pasos intermedios y una estructura que las herramientas

ofimáticas no facilitan.

5.1 ¿Qué es OpenCode?

Antes de continuar, conviene saber con qué vamos a trabajar. OpenCode es un **asistente de inteligencia artificial** que se ejecuta en tu ordenador y transforma archivos:

Pregunta	Respuesta
¿Qué hace?	Transforma archivos de un formato a otro usando lenguaje natural
¿Es gratuito?	Sí
¿Necesita instalación?	Sí, pero el taller guiado lo instala en minutos
¿Necesita internet?	Depende. Con un modelo remoto, sí. Con un modelo local, no
¿Funciona en Windows, Mac, Linux?	En los tres
¿Sube mis archivos a la nube?	Depende del modelo. Con un modelo local, no. Con un modelo remoto, los envía al proveedor para procesarlos, pero no se almacenan permanentemente

Para más detalles, consulta la [presentación del taller](#) o mira el [vídeo de la sesión](#).

5.2 Ejemplo: de un borrador a un informe

Para que veas cómo funciona en la práctica, supongamos que tienes un borrador como este:

i Tu borrador (Markdown)

```
# Informe de prácticas - Biología

## Objetivo

Evaluar el crecimiento de tres muestras de *Arabidopsis*
bajo condiciones de estrés hídrico.

## Resultados

| Muestra | Crecimiento |
|-----|-----|
| A       | 12%         |
| B       | 8%          |
| C       | Sin datos   |

## Conclusiones

La muestra A muestra mayor resistencia al estrés.
La muestra C requiere una segunda ronda de mediciones.
```

Ahora le pides a OpenCode: “*Genera un informe en PDF a partir de este texto, con estilo académico y tabla de contenidos*”. El resultado es un documento formateado, con portada, numeración de páginas y tabla de contenidos automática. Sin tocar Word. Sin ajustar estilos. Sin perder tiempo maquetando.

Una vez que tienes tu contenido en texto plano, un **agente** —un programa de inteligencia artificial como OpenCode— se encarga de transformarlo al formato que necesites: un documento de Word, una presentación, una página web, un PDF, un correo ... todo desde el mismo texto original.

El cambio es sutil pero profundo:

! Importante

Modelo actual: Abres el programa → creas el contenido dentro de él → el formato es parte del programa → para otro formato, rehaces.

Modelo nuevo: Escribes en texto plano → le pides al agente que lo transforme → obtienes el formato que necesitas → para otro formato, vuelves a transformar, no a reescribir.

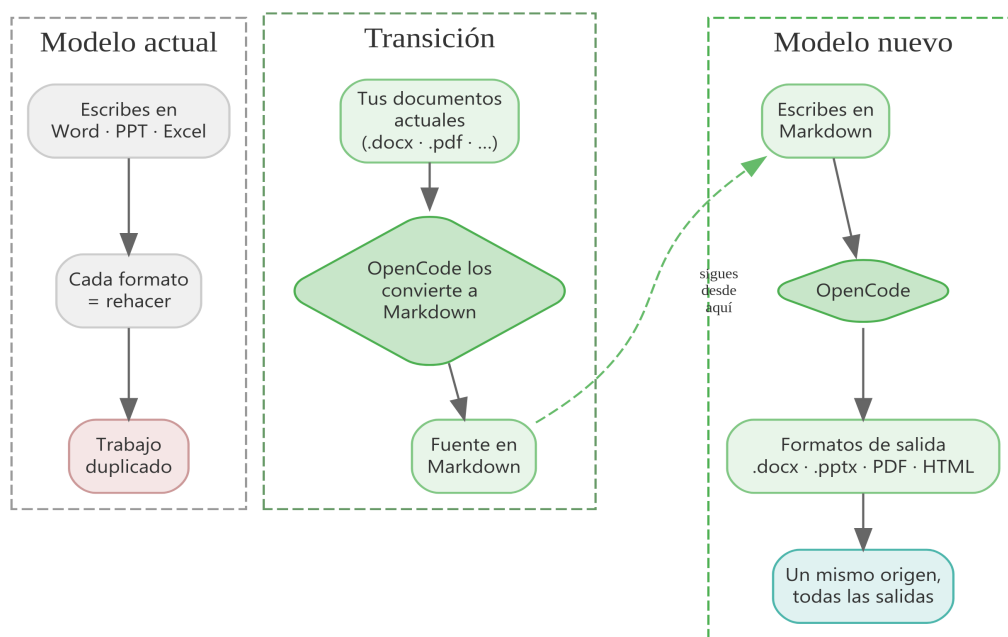


Figura 5.1: Comparación entre el modelo actual, la transición y el modelo nuevo

La clave está en que **el contenido y la presentación ya no van ligados**. Tu texto original es la fuente de verdad. A partir de él generas las versiones que hagan falta. Y no necesitas saber cómo se usa PowerPoint para obtener una presentación: se la pides al agente.

Esto no significa que nunca más toques Word o PowerPoint. Significa que dejas de depender de ellos. Los usas cuando toca, pero ya no eres su prisionero.

Hay un matiz importante que a menudo se pasa por alto. El formato en el que tú trabajas y el formato que necesita quien recibe el documento no tienen por qué ser el mismo. De hecho, no deberían serlo.

Como autor, te interesa un formato que sea fácil de escribir, fácil de corregir, fácil de versionar. Eso es el texto plano: lo abres con cualquier editor, cambias lo que quieras y guardas. Sin menús, sin estilos que se corrompen, sin esperar a que cargue el programa.

Como lector, lo que necesitas es un documento bien presentado, en el formato que exige tu contexto: el PDF para la revista, el .docx para el departamento, la presentación para el seminario. Eso no lo discute nadie.

OpenCode es el puente entre ambos mundos. Tú trabajas en el formato que te va bien a ti. OpenCode se encarga de generar el formato que le va bien a quien lo recibe. Y lo hace sin perderse: combinando fuentes si hace falta, aplicando plantillas, garantizando que el resultado final es exactamente lo que pediste. Revisas, confirmas, entregas.

6 Qué ganas con el cambio

Pasar a este modelo no es solo una cuestión de principios. Tiene beneficios muy concretos:

6.1 Calidad

- **Menos errores de formato.** No se te va un título porque lo escribiste en el estilo equivocado. El agente aplica las plantillas correctamente.
- **Consistencia.** Si cambias algo en el texto original, puedes regenerar todos los formatos de salida con una sola instrucción. No se te queda una versión antigua colgando.
- **Contenido mejor estructurado.** Markdown te obliga a pensar en jerarquías: qué es un título, qué es un subtítulo, qué es una lista. Eso mejora la claridad de lo que escribes.

6.2 Tiempo

- **No pierdes horas maquetando.** El agente se encarga de los ajustes finos de formato. Tú te centras en lo que quieres decir.
- **No reescribes lo mismo.** Una vez que tienes el texto, generas todos los formatos que necesites desde el mismo origen.
- **Tareas repetitivas delegadas.** Cambiar el formato de cien títulos, ajustar la numeración de las páginas, convertir tablas ... se lo pides al agente y lo hace en segundos.

Tip

El cambio no es binario. Puedes mantener Word para los documentos que ya tienes en marcha y, para los nuevos, probar a escribirlos en Markdown. Pronto descubres que las herramientas ofimáticas no te ayudan a estructurar bien la información; el texto plano, en cambio, te obliga a pensar en jerarquías limpias. Y eso se nota en la claridad del resultado final.

6.3 Aprendizaje

- **Piensas mejor lo que pides.** Al tener que formular tus peticiones con claridad —acción, contexto, formato—, aprendes a ser más preciso. Eso mejora tus habilidades de comunicación en general.
- **Ves cómo lo hace.** OpenCode te muestra lo que está haciendo. Puedes aprender de sus estrategias y aplicarlas tú mismo más adelante.
- **Reutilizas lo que funciona.** Cuando encuentras una forma de pedir que da buenos resultados, la guardas y la usas de nuevo. Creas tu propio repertorio de instrucciones efectivas.

6.4 Independencia

- **No dependes de una suite concreta.** Tus documentos no caducan con el programa. El texto plano se lee siempre, en cualquier ordenador, ahora y dentro de veinte años.
- **Puedes cambiar de herramienta.** Si mañana dejas de usar Word y pasas a otra cosa, tu contenido sigue siendo el mismo. No pierdes nada.
- **Tus archivos están en tu ordenador.** No necesitas copiarlos a una plataforma externa para que los procese el agente. OpenCode trabaja sobre tus archivos directamente —aunque para pensar necesite conexión a internet, no almacena tus datos en ningún servidor externo de forma permanente.

i Nota

Tus versiones, siempre a salvo. Como trabajas con texto plano, cada cambio que haces queda registrado: no se pierde nada. OpenCode se encarga de gestionar el proyecto y de mantener el historial de versiones que dejas con cada paso. Puedes volver atrás, comparar, retomar una idea anterior. Sin tener que guardar «informe_v3_final_definitivo.docx».

El cambio no es solo tecnológico: es un cambio de mentalidad. Pasas de ser **usuario de programas** a ser **creador de contenido asistido**.

7 De la teoría a la práctica

Este documento es la base conceptual. Describe el problema y dibuja la dirección del cambio. Pero el cambio se aprende haciendo.

Los talleres prácticos —como la sesión *Asistente IA para el trabajo universitario*— te guían paso a paso para que experimentes este modelo con tus propios proyectos. Ahí verás cómo se escribe en Markdown, cómo se le pide a OpenCode que transforme contenido, cómo se revisa el resultado y cómo se itera hasta obtener lo que buscas.

Dos documentos complementarios te ayudarán a situarte:

- **Diagnóstico del perfil de usuario** — Describe quién es el creador de contenidos universitario, sus barreras personales y culturales, su relación con la tecnología y los errores típicos al empezar.
- **Presentación del taller** — Material de la sesión práctica con ejemplos, ejercicios y referencias para seguir aprendiendo.

Prueba esto ahora

Si quieres una idea rápida de cómo funciona este modelo, abre el Bloc de Notas (o cualquier editor de texto) y escribe estas tres líneas:

```
# Mi primer documento
```

```
Este es un **párrafo de prueba** con negrita.
```

Ahora abre OpenCode y dile: “*Convierte este texto a PDF*”. En segundos verás un documento formateado con título, párrafo y negrita. Sin tocar Word. Sin ajustar estilos.

Si no tienes OpenCode instalado, mira el [vídeo de la sesión](#) para ver cómo se hace en la práctica.

7.1 Versionado: olvídate de «versión_final_corregida.docx»

Si trabajas con documentos —informes, guías, actas, presentaciones—, seguramente reconocas estos nombres:

- informe_v2.docx
- informe_v2_corregido.docx
- informe_v2_corregido_definitivo.docx
- informe_v2_corregido_definitivo_REAL.docx

Son intentos de resolver un problema real: **¿cómo guardo las versiones anteriores de un documento sin que se acumulen y me estorben?** La respuesta habitual es crear copias con nombres cada vez más largos. Pero eso no es un control de versiones: es un desastre. No puedes saber qué cambió entre una versión y otra. No puedes volver atrás sin abrir diez archivos. Y si alguien modifica la versión equivocada, el lío está servido.

7.1.1 Lo que el versionado resuelve

El versionado —usando herramientas como Git— te permite:

- **Guardar cada cambio** con un mensaje que explica qué se hizo y por qué.
- **Volver atrás** a cualquier punto del historial, sin buscar entre copias.
- **Comparar versiones** para ver exactamente qué cambió, línea por línea.
- **Trabajar en paralelo** sin temor a sobrescribir el trabajo de otro.
- **Archivar versiones anteriores** sin que estorben: están registradas, pero no ocupan espacio en tu escritorio.

7.1.2 Cómo funciona con OpenCode

Cuando trabajas con OpenCode sobre un proyecto, el historial de versiones se gestiona automáticamente. Cada vez que el agente modifica un archivo, puedes guardar el cambio con un mensaje descriptivo. No necesitas crear copias manualmente: el sistema las lleva por ti.

Si necesitas recuperar algo que se borró o comparar cómo era un documento antes de una edición, lo haces con una instrucción sencilla. Sin buscar en carpetas, sin abrir archivos uno por uno.

Tip

En la práctica. Imagina que tienes un borrador de una guía docente. Le pides a OpenCode que reorganice las secciones, que añada ejemplos y que adapte el tono. Si el resultado no te convence, puedes volver al estado anterior con un solo comando. Si te convence, guardas el cambio con un mensaje como «Reorganización de secciones y adición de ejemplos». Mañana, o dentro de un mes, podrás ver qué hiciste y por qué. Sin «versión_final_corregida_v3.docx».

El versionado no es una herramienta solo para programadores. Es una forma de **trabajar con documentos de verdad**: con historia, con seguridad y sin miedo a equivocarte.

8 Conclusión

El modelo de trabajo que la mayoría de la universidad da por sentado —abrir un programa, escribir, formatear, guardar— no es el único posible. Y tiene un coste que apenas percibimos porque lo heredamos sin cuestionarlo.

Hay otra forma de trabajar. Una donde el contenido no depende de la aplicación que lo creó. Donde un agente de inteligencia artificial se encarga de las transformaciones. Donde no necesitas ser experto en cada herramienta para obtener buenos resultados.

No se trata de tecnología por la tecnología. Se trata de **recuperar el control sobre tu propio contenido**. Dejar de preguntarte «¿cómo se hace esto en Word?» y empezar a preguntarte «¿qué necesito comunicar y a quién?».

El cambio empieza por entender el modelo. Sigue por probarlo con algo pequeño. Y se consolida cuando ves que funciona y no quieres volver atrás.

Referencias

8.1 Para empezar

Recursos prácticos y accesibles para familiarizarte con el modelo de trabajo:

- **Presentación del taller** — Material visual con ejemplos y ejercicios guiados.
- **Vídeo de la sesión explicativa** — Grabación de una sesión en la que se demuestra el flujo de trabajo completo.
- **Marcando Palabras** — Libro en desarrollo con ejercicios prácticos de Markdown y Quarto. Próximamente disponible.
- **Guía rápida de Markdown** — Referencia concisa de la sintaxis básica (títulos, negritas, listas, enlaces).
- **Glosario** — Definición de los términos técnicos usados en estos documentos.
- **Preguntas frecuentes** — Resolución de dudas prácticas antes de lanzarte a probar.

8.2 Para profundizar

Textos académicos y comunitarios que profundizan en la filosofía y la técnica detrás de este modelo:

- **Escritura sostenible en texto plano usando Pandoc y Markdown** Dennis Tenen y Grant Wythoff — *Programming Historian* (trad. al español)

Guía académica de referencia sobre el modelo de trabajo con texto plano. Explica por qué escribir en texto plano libera al investigador de formatos propietarios y frágiles, y cómo Pandoc permite generar cualquier formato de salida desde un mismo origen.

- **Blog de Cibercliografía — escritura académica sostenible** Varios autores

Recopilación de artículos, reseñas y ampliaciones en torno a la escritura académica en texto plano. Incluye comentarios y aplicaciones prácticas del artículo de Tenen.

- **The Art of UNIX Programming** Eric S. Raymond — Addison-Wesley, 2003

Clásico de la filosofía Unix que explica los principios de hacer una sola cosa bien, conectar programas mediante tuberías de texto y mantener la simplicidad.

- **Reading Machines: Toward an Algorithmic Criticism** Stephen Ramsay — University of Illinois Press, 2011

Reflexión sobre cómo la computación cambia la forma de leer y escribir. Ramsay defiende que el ordenador no solo procesa texto, sino que abre nuevas formas de interpretación.

- **Write the Docs** Comunidad global

Comunidad de referencia en el mundo de la documentación técnica. Su filosofía —documentación como código, separación de contenido y presentación— es paralela al modelo que aquí se presenta.

- **What Is Code?** Paul Ford — Bloomberg Businessweek, 2015

Ensayo divulgativo que explica qué es el código y cómo el texto plano se convierte en programas. Una lectura complementaria excelente para entender por qué separar contenido de forma es una idea tan potente.

Anexo A: Las plantillas universitarias y el modelo actual

Las plantillas ofimáticas que circulan en la universidad —las que usas para artículos, tesis, formularios— comparten un patrón preocupante. Un análisis de diez de ellas revela que la mayoría dependen del formato directo (aplicar estilo párrafo a párrafo con los botones de la barra) en lugar de usar estilos de párrafo. Esto significa que cada cambio de formato debe hacerse a mano, una y otra vez, en cada sección, en cada documento.

Los problemas más habituales son: estilos declarados pero que nadie usa, instrucciones mezcladas con el contenido real, ausencia de tabla de contenidos automática y falta de control de versiones. En el extremo opuesto, alguna plantilla logra un 99 % de cobertura de estilos y demuestra que, cuando la estructura está bien pensada, el procesador de textos funciona razonablemente bien.

La conclusión es clara: **el problema no es la herramienta, sino el flujo de trabajo y la formación de quien diseña la plantilla**. Un mismo programa (Word) alberga desde documentos con formato 100 % manual hasta documentos con estructura casi perfecta.

Si quieres ver el análisis completo —con datos, tablas y los cuatro patrones identificados—, consulta estos documentos:

- **Ejemplo de uso ineficiente de un procesador de textos** — Caso real de dos plantillas que reproducen todas las ineficiencias descritas. Ver [documento completo](#).
- **Características compartidas en el uso del procesador de textos** — Análisis comparativo de diez documentos universitarios con datos cuantitativos y patrones transversales. Ver [documento completo](#).